

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

CÁC VẤN ĐỀ HIỆN ĐẠI CỦA CÔNG NGHỆ THÔNG TIN



HỆ THỐNG PHÒNG THỬ DNN TRONG
PHÂN LOẠI ẢNH

Lớp : INT3507 7
Giảng viên hướng dẫn : TS. Nguyễn Đức Anh
Nhóm : 10
Thành viên : Dương Thị Thu Thảo - 22022638
Đỗ Thị Thùy Trang - 22022617
Nguyễn Phương Trang (Nhóm trưởng) - 22022656
Cao Xuân Nguyên - 22022670

STT	Họ và tên	MSV	Phân công	Đóng góp
1	Nguyễn Phương Trang	22022656	- Tìm hiểu puVAE - Cài đặt mô hình PuVAE - Báo cáo kết quả cài đặt	25%
2	Dương Thị Thu Thảo	22022638	- Tìm hiểu PuVAE - Tổng hợp báo cáo + làm slide - Báo cáo thực nghiệm + giới thiệu	25%
3	Đỗ Thị Thuỳ Trang	22022617	- Tìm hiểu PuVAE - Tìm hiểu các bộ phòng thủ khác - Báo cáo phương pháp và tổng quan	25%
4	Cao Xuân Nguyên	22022670	- Tìm hiểu PuVAE - Tìm hiểu các bộ tấn công - Cài đặt bộ tấn công cho puVAE	25%

Bảng 1: Mô tả phân công công việc

Lời cam đoan

Báo cáo này được viết dưới sự hướng dẫn của thầy Nguyễn Đức Anh - giáo viên phụ trách môn Các vấn đề hiện đại của Công nghệ thông tin. Chúng em xin cam đoan bài báo cáo này hoàn toàn do chúng em viết mà không sử dụng sự trợ giúp của các công nghệ AI như chatGPT.

Người viết

Nhóm 10

Lời cảm ơn

Trước tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến thầy giáo bộ môn Các vấn đề hiện đại của Công nghệ thông tin - thầy Nguyễn Đức Anh, người đã hướng dẫn, chỉ dạy, cũng như đưa ra những lời khuyên hữu ích cho chúng em.

Trong quá trình hoàn thiện báo cáo, nhóm em đã cố gắng tìm hiểu, cũng như cài đặt mô hình sát với bài báo nhất có thể, đồng thời tham khảo các tài liệu liên quan. Tuy nhiên, do thời gian có hạn cộng thêm việc hạn chế về mặt tài nguyên nên chúng em sẽ không tránh khỏi những thiếu sót, rất mong nhận được sự đóng góp ý kiến của thầy và các bạn để báo cáo và kỹ năng của chúng em ngày càng hoàn thiện hơn.

Mục lục

1	Tóm tắt nội dung	1
2	Giới thiệu	1
3	Tổng quan	8
3.1	Variational Autoencoder (VAE)	8
3.2	Adversarial Example	9
3.2.1	Fast Gradient Sign Method (FGSM)	9
3.2.2	Iterative Fast Gradient Sign Method (iFGSM)	10
3.2.3	Randomized Fast Gradient Sign Method (RAND+FGSM)	11
3.2.4	Carlini-Wagner attack (CW attack)	11
4	Phương pháp được đề xuất	12
4.1	Quá trình huấn luyện PuVAE	13
4.2	Quá trình sinh mẫu đã được thanh lọc	14
5	Thực nghiệm	16
5.1	Ảnh hưởng của hệ số trong quá trình đào tạo PuVAE	17
5.2	Hiệu suất phòng thủ	18
5.3	Hiệu suất trong giới hạn thời gian hợp lý	20
6	Kết luận	22
7	Phụ lục	23
7.1	Kết quả cài đặt	23
7.2	Hiệu suất của mô hình khi tấn công đối kháng theo phân phối chuẩn	23
7.3	Hiệu suất của mô hình khi dữ liệu có kích thước lớn	24

Danh sách hình vẽ

1	Tổng quan cơ chế phòng thủ sử dụng mô hình purifier.	2
2	Tổng quan về mô hình GAN	6
3	Tổng quan phương pháp được đề xuất.	13
4	Quá trình suy luận của PuVAE với ảnh MNIST.	14
5	Kiến trúc mạng nơ-ron của PuVAE	17
6	Kiến trúc mạng nơ-ron dùng cho bộ phân loại mục tiêu.	17
7	So sánh các hệ số trong quá trình training PuVAE.	18
8	Hiệu suất phòng thủ trên tập dữ liệu MNIST.	18
9	Hiệu suất phòng thủ trên tập dữ liệu Fashion-MNIST.	19
10	Hiệu suất phòng thủ trên tập dữ liệu CIFAR10.	19
11	Hiệu suất phòng thủ trên tập dữ liệu Fashion-MNIST.	20
12	So sánh thời gian suy luận của PuVAE và Defense-GAN.	20
13	So sánh thời gian suy luận của PuVAE và Defense-GAN.	21
14	Độ chính xác của phương pháp defense trong một giây.	22
15	Kết quả cài đặt cũ	23
16	Kết quả cài đặt mới	23
17	Chỉ số SSIM	23
18	Kết quả của ảnh tái tạo trên bộ dữ liệu STL10	24

1 Tóm tắt nội dung

Hiện nay, các mạng nơ-ron sâu đang được ứng dụng rộng rãi và đạt hiệu suất vượt trội trong nhiều lĩnh vực như hệ thống nhận diện hình ảnh, hệ thống dự đoán và xe tự hành. Tuy nhiên, chúng dễ bị ảnh hưởng bởi các cuộc tấn công đối kháng (adversarial attacks), ở đó các ảnh đầu vào bị thêm nhiễu tinh vi khiến mạng nơ-ron đưa ra dự đoán sai lệch. Mặc dù đã có nhiều phương pháp phòng thủ được phát triển để chống lại các cuộc tấn công này nhưng đa phần chúng chỉ hiệu quả với một số dạng tấn công nhất định và dễ bị đánh lừa khi đối mặt với các thuật toán tấn công mới.

Để giải quyết vấn đề này, trong bối cảnh bài báo được xuất bản, tác giả đề xuất PuVAE (Purifying Variational Autoencoder) [7], phương pháp thanh lọc mẫu đối kháng. PuVAE sử dụng mạng Variational Autoencoder (VAE) chiếu mẫu đối kháng lên một không gian ẩn và chọn ra điểm chiếu gần nhất làm mẫu “sạch” cho mạng nơ-ron phân loại.

Qua thực nghiệm, PuVAE cho thấy khả năng chống lại hiệu quả với nhiều phương pháp tấn công khác nhau mà không biết trước đặc điểm của các cuộc tấn công. Ngoài ra, PuVAE có thể tạo ra mẫu đã thanh lọc nhanh hơn khoảng 130 lần so với Defense-GAN [13], điều này phù hợp với các ứng dụng thời gian thực.

2 Giới thiệu

Học sâu đang là một trong những kỹ thuật nổi bật nhất hiện nay do khả năng vượt trội khi xử lý các bài toán phức tạp. Trong nhận diện hình ảnh, học sâu gần như có thể nhận diện đối tượng ngang với con người. Hay trong lĩnh vực y tế, mạng nơ-ron sâu có thể phân tích lượng lớn dữ liệu và chẩn đoán sớm các dấu hiệu của bệnh.

Tuy nhiên, các mạng nơ-ron sâu vẫn gặp phải thách thức bảo mật đến từ các cuộc tấn công đối kháng. Các cuộc tấn công này được thực thi bằng cách thêm nhiễu một cách tinh vi vào dữ liệu nhằm đánh lừa mô hình, khiến mô hình dự đoán sai. Những dự đoán sai này có thể gây ra hiệu quả nghiêm trọng. Ví dụ, khi ứng dụng vào hệ thống xe tự lái, nếu mô hình nhận diện nhầm người đi đường thành mặt đường thì có thể dẫn đến tai nạn [1].

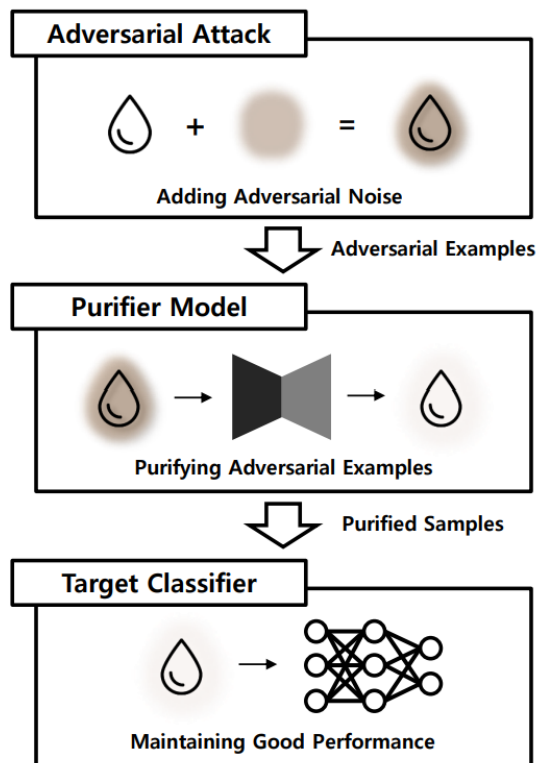
Để giải quyết vấn đề này, có ba hướng tiếp cận chính đã được đề xuất:

1. Điều chỉnh dữ liệu huấn luyện.
2. Chặn việc tính toán gradient.
3. Loại bỏ nhiễu đối kháng.

Trong nghiên cứu này, nhóm tác giả tập trung vào hướng thứ ba là thanh lọc các dữ liệu đầu vào có thể bị thêm nhiễu đối kháng. Phương pháp phòng thủ này có tính toàn diện và hiệu quả trong việc xử lý nhiều dạng tấn công đối kháng khác nhau. Các phương pháp phòng thủ cũ thường chỉ hiệu quả đối với một số loại tấn công nhất định. Tuy nhiên, PuVAE có thể xử lý đa dạng các loại nhiễu đối kháng do nó tập trung vào việc

đưa dữ liệu về phân phối gốc thay vì chỉ dựa vào đặc điểm của từng cuộc tấn công cụ thể.

Cơ chế phòng thủ này thường sử dụng các mô hình sinh để học phân phối dữ liệu gốc, sau đó chiếu mẫu đối kháng về phân phối đã học $p(x)$, những mô hình sinh này được gọi là *purifier*.



Hình 1: Tổng quan cơ chế phòng thủ sử dụng mô hình purifier.

Hình 1 trình bày khái quát về cơ chế phòng thủ bằng cách sử dụng mô hình thanh lọc (purifier model). Đầu tiên, nhiễu đối kháng được thêm vào dữ liệu đầu vào để tạo ra mẫu đối kháng, nhằm đánh lừa mô hình. Sau đó, mẫu đối kháng được đưa qua mô hình thanh lọc (purifier model) để loại bỏ nhiễu và trả về mẫu sạch gần với dữ liệu gốc. Cuối cùng, mẫu đã thanh lọc được đưa vào bộ phân loại mục tiêu để đảm bảo phân loại chính xác và duy trì hiệu suất tốt.

Magnet [12] là một khung phòng thủ được đề xuất để bảo vệ các mô hình học sâu, đặc biệt là các hệ thống sử dụng mô hình phân loại hình ảnh. Hướng tiếp cận của Magnet là cải thiện nhiễu đối kháng có thể xuất hiện trong bộ dữ liệu. Magnet được coi là một kỹ thuật sàng lọc nhiễu linh hoạt vì không tác động vào kiến trúc ban đầu của mô hình phân loại cũng như không can thiệp vào quá trình huấn luyện của hệ thống. Điều này giúp giảm thiểu nguy cơ gây lỗi cho bộ phân loại, đồng thời tránh tình trạng chòng chẹo hệ thống và gây nhiễu loạn thông tin.

Có hai lý do chính dẫn đến việc một bộ phân loại bình thường không thể phân loại một mẫu đối kháng:

1. Mẫu đó cách xa so với biên của đa tạp dữ liệu (*data manifold*). Điều này có nghĩa là mẫu đối kháng không nằm hoặc không thuộc vào bất kỳ phân phối nào xuất hiện trong tập dữ liệu. Khi đó, bộ phân loại không thể xác định nhãn tương ứng cho mẫu đó và cũng không có lựa chọn loại bỏ mẫu. Do đó, hệ thống sẽ có xu hướng gán nhãn cho mẫu đó giống với nhãn của các dữ liệu nằm ở sát biên của *data manifold*.
2. Mẫu nằm gần hoặc nằm trên *data manifold*, tuy nhiên việc xác định nhãn cho mẫu đó gặp khó khăn. Nguyên nhân có thể là do các xác suất thể hiện không rõ ràng theo quan sát, hoặc mẫu có thể tuân theo nhiều hơn một phân phối trong dữ liệu. Điều này dẫn đến việc tổng quát hóa kém của mô hình, từ đó gây ra dự đoán sai nhãn của dữ liệu.

Với các lý do trên, MagNet được xây dựng với hai thành phần chính, được triển khai dưới dạng mạng nơ-ron học sâu. Bộ phòng thủ này tận dụng kiến trúc của hệ thống học sâu phổ biến là Autoencoders — một loại mạng nơ-ron nhân tạo được sử dụng để học cách mã hóa (*encode*) dữ liệu đầu vào thành dạng biểu diễn khái quát hơn, sau đó tái tạo (*decode*) dữ liệu ban đầu dựa trên biểu diễn đã được mã hóa. MagNet bao gồm một hoặc nhiều hệ thống phát hiện nhiễu (*detector*) độc lập với nhau và một bộ cải thiện (*reformer*) có kiến trúc kế thừa từ Autoencoders.

Một bộ autoencoder học các đặc trưng của dữ liệu được huấn luyện sao cho encoder có thể mã hóa đầu vào với các đại diện ẩn của một số đặc điểm nhất định. Trong khi đó, bộ decoder đảm nhiệm công việc tái cấu trúc dữ liệu đầu vào dựa vào các đặc điểm đã được mã hóa. Đây chính là ý tưởng chính của hệ thống autoencoder cho tất cả các ứng dụng liên quan đến kiến trúc này. Với khả năng học và tái tạo dữ liệu đầu vào mạnh mẽ, việc tận dụng khả năng này sẽ giúp bộ phòng thủ có khả năng nhận diện được các mẫu đối kháng và tái tạo, chỉnh sửa chúng một cách dễ dàng hơn so với việc nhận dạng bằng mắt thường rất nhiều.

Bộ phát hiện nhiễu sử dụng Autoencoder để ước lượng sai khác giữa đầu vào so với dữ liệu thông thường bằng phép đo khoảng cách đặc biệt từ đầu vào đến mặt phẳng được tạo bởi dữ liệu bình thường. Cụ thể, autoencoder được huấn luyện trên dữ liệu không bị nhiễu để học trước các đặc trưng của dữ liệu và hình thành các khái niệm về mặt phẳng của dữ liệu sạch. Sau khi hoàn thành huấn luyện và đưa vào sử dụng, các mẫu trong dữ liệu sẽ được đưa qua các bộ phát hiện để thực hiện phép đo đến mặt phẳng dữ liệu sạch đã được hình thành. Nếu một mẫu có giá trị đo được vượt ngưỡng xác định, mẫu đó được coi là mẫu đối kháng. *Detector* được định nghĩa về mặt toán học là một hàm $d : S \rightarrow \{0, 1\}$ mà quyết định một đầu vào có mẫu đối kháng hay không. Có một số loại detector có thể lựa chọn để tối ưu bài toán trong điều kiện cụ thể như:

- Detector dựa trên lỗi tái tạo (reconstruction error): Cách tiếp cận này phát hiện mẫu là mẫu đối kháng khi dữ liệu của mẫu bị chỉnh sửa quá nhiều sau khi đi qua các bộ autoencoder. Việc xác định xem mẫu có bị chỉnh sửa nhiều hay không được xác định dựa trên reconstruction error, là độ sai khác của đầu ra của autoencoder so với đầu vào được nhận được tính theo công thức cho một mẫu thử x như sau:

$$E(x) = ||x - ae(x)||_2$$

Trong đó:

- $ae = d \circ e$: Là một bộ autoencoder chứa hai thành phần: $e : S \rightarrow H$ là encoder và $d : H \rightarrow S$ là decoder với S là không gian đầu vào và H là không gian tiềm ẩn (hidden representation)
- $E(x)$: Hàm lỗi tái tạo thu được của mẫu x sau khi huấn luyện autoencoder bằng cách tối ưu hàm mất mát tương ứng của bộ dữ liệu huấn luyện

Nếu một mẫu được tính toán có lỗi tái tạo $E(x)$ lớn, vượt qua một ngưỡng đầu vào nhất định được đặt như một siêu tham số, mẫu này được xác định là mẫu chứa nhiễu đối kháng. Ngược lại, khi giá trị này mang giá trị nhỏ, tức là mẫu này dường như nằm trong data manifold của dữ liệu huấn luyện, chứng tỏ đầu vào không bị tác động quá nhiều nên có thể xác định đây là một đầu vào bình thường.

Detector được triển khai theo phương án này tỏ ra hiệu quả đối với các mẫu được thêm nhiễu rõ rệt, hay $E(x)$ mang giá trị lớn

- Detector dựa trên sự phân biệt xác suất (probability divergence): Đối với các trường hợp nhiễu thêm vào mẫu đối kháng ở mức nhỏ, tận dụng lợi thế của hệ thống phân loại mục tiêu (target classifier) như sau:
 - Đa số kiến trúc của mạng phân loại đều định nghĩa một hàm softmax-hàm trung bình mũ:

$$\text{softmax}(\mathbf{l})_i = \frac{\exp(l_i)}{\sum_{j=1}^n \exp(l_j)}$$

- Đầu ra của *softmax* là một hàm khối xác suất tương ứng với từng lớp có trong dữ liệu. Hàm này nhận giá trị đầu vào chính là đầu ra của lớp kế cuối trong mạng nơ-ron phân loại dưới dạng vectơ có tên là logit $\mathbf{l}$. Định nghĩa $\text{rank}(\mathbf{l}, i)$ là chỉ số tương ứng với phần tử lớn thứ i trong logit $\mathbf{l}$ thuộc một mẫu ví dụ, mục tiêu của bộ tấn công là gây nhiễu mẫu để tạo ra một logit mới $\mathbf{l}'$ sao cho $\text{rank}(\mathbf{l}, 1) \neq \text{rank}(\mathbf{l}', 1)$. Hay gây nhiễu đối kháng sao cho mẫu đầu vào không được nhận nhận có xác suất cao nhất mà mẫu đó nên thuộc về.

Đặt $f(x)$ là đầu ra của lớp cuối cùng của mạng f trên dữ liệu x . Nếu x là mẫu bình thường, tức là $ae(x)$ rất gần với x thì hàm khối xác suất $f(ae(x))$ và $f(x)$ là tương đồng nhau. Ngược lại, nếu x' là mẫu đối kháng thì $ae(x')$ sẽ khác biệt rõ rệt so với x' . Vậy nên kể cả khi lỗi tái tạo $E(x')$ của x nhỏ thì hàm khối xác suất $f(x')$ và $f(ae(x'))$ có thể khác nhau rất nhiều. Điều này chứng tỏ rằng sự sai khác giữa $f(x)$ và $f(ae(x))$ phản ánh khả năng x đến từ một bộ sinh dữ liệu thông thường hay không.

Bộ cải thiện, cũng dựa trên autoencoder, có chức năng điều chỉnh các ví dụ đối kháng sao cho chúng trở về với hình ảnh bình thường hơn. Điều này có nghĩa là điều chỉnh sao cho khoảng cách của hình ảnh nhiễu đến mặt phẳng dữ liệu sạch gần nhất có thể. Sau khi thực hiện, hình ảnh ban đầu bị phát hiện có chứa nhiễu, giờ đây được coi là đã được lọc nhiễu, sẽ được sử dụng cho hệ thống phân loại. Như vậy, mặc dù ảnh có thể

bị nhiễm nhiễu đối kháng, nó vẫn có thể được phân loại chính xác bởi mô hình phân loại với xác suất chính xác được cải thiện, có thể ở mức tốt hơn nhiều so với ảnh thật được thêm nhiễu đối kháng. Bộ cải thiện được định nghĩa đơn giản về mặt toán học là một hàm $r : S \rightarrow N_t$, có tác dụng tái tạo đầu vào. Đầu ra của reformer sau đó được đưa vào bộ phân loại mục tiêu. Chúng ta không dùng reformer khi huấn luyện bộ phân loại mục tiêu nhưng khi dùng triển khai bộ phân loại thì sẽ dùng reformer. Một bộ cải thiện lí tưởng phải thoả mãn:

1. Phải không làm thay đổi các kết quả phân loại của mẫu bình thường
2. Nên thay đổi các mẫu đối kháng phù hợp để mẫu được tái tạo sát với mẫu bình thường. Hay nói cách khác, nó sẽ cải thiện các mẫu đối kháng

Với hướng tiếp cận xây dựng Reformer bằng autoencoder (autoencoder - based reformer), mục tiêu là huấn luyện mạng autoencoder để có thể cải thiện hình ảnh bằng cách tối thiểu hóa hàm lỗi tái tạo trên tập dữ liệu huấn luyện và đảm bảo rằng khả năng sinh mẫu đạt tốt trên tập đánh giá (validation set). Một bộ autoencoder đã đủ tốt là khi cho mẫu bình thường giống như trong bộ huấn luyện thì autoencoder cho ra một đầu ra phải vô cùng tương tự với đầu vào. Đồng thời, khi đầu vào là mẫu đối kháng, autoencoder phải cho ra đầu ra tương ứng sao cho đầu ra đó thuộc gần hơn vào data manifold của mẫu bình thường.

Dựa trên thực nghiệm được thực hiện trong [12], Magnet được triển khai để chống lại một số cuộc tấn công mô hình sử dụng các phương pháp tạo mẫu đối kháng khác nhau trên nhiều tập dữ liệu khác nhau và cho ra các kết quả thực nghiệm có tính cạnh tranh với các bộ phòng thủ khác. Các kết quả nêu ra rằng Magnet có thể được sử dụng như một bộ phòng thủ cho các hệ thống phân loại sử dụng mạng nơ-ron học sâu hiện tại. Tuy nhiên, hệ thống này còn biểu hiện một vài hạn chế về mặt tối ưu cũng như hiệu suất. Magnet hiện tại đang phụ thuộc rất nhiều vào phần detector để phát hiện ra mẫu đối kháng, và trong nhiều trường hợp nếu không điều chỉnh phù hợp, detector sẽ phân biệt sai hoặc bỏ qua cho các mẫu đối kháng dẫn đến giảm thiểu tỉ lệ chính xác cho reformer. Bên cạnh đó, vấn đề tạo sinh cũng là một thách thức khi Magnet không yêu cầu về kiến trúc để sinh ra các mẫu đối kháng.

Magnet là một bộ phòng thủ có cơ chế cũng như kiến trúc sử dụng các bộ autoencoder để thực hiện cải thiện hình ảnh và đưa ảnh về gần với data manifold của dữ liệu sạch. Tuy có các số liệu thực nghiệm khả quan, tuy nhiên Magnet vẫn cần thời gian để nghiên cứu và phát triển thêm để tối ưu hoá hiệu suất, phát hiện được nhiều tình vi hơn.

Defense-GAN [13] là một bộ phòng thủ được đề xuất sử dụng mô hình sinh (Generative model). Một mô hình là mô hình sinh khi mô hình học máy đó có khả năng học được phân phối của dữ liệu ban đầu đồng thời tạo ra được các mẫu mới dựa trên phân phối đó. Một ví dụ về mô hình sinh đã nêu ở trên chính là mạng autoencoder được sử dụng trong bộ phòng thủ Magnet. Defense-GAN sử dụng ý tưởng và kiến trúc của mô hình mạng tạo sinh đối kháng GAN (Generative Adversarial Network) [5] đồng thời đề xuất sử dụng biến thể WGAN của GAN để thực hiện việc chống lại các cuộc tấn công khác nhau.

GAN bao gồm một cặp mạng phức tạp có quan hệ ràng buộc với nhau là mạng sinh (generator) và mạng phân biệt (discriminator). Trong quá trình huấn luyện, GAN tối ưu hóa hàm mục tiêu nhằm làm cho phân phối của dữ liệu sinh ra từ generator càng giống với dữ liệu thật càng tốt. Sau khi quá trình huấn luyện hoàn tất, generator có thể sinh ra dữ liệu giả nhưng có các đặc trưng tương tự như dữ liệu gốc.

Mạng sinh, hay generator, là phần đầu của GAN, có tác dụng sinh ra mẫu giả giống với dữ liệu thật. Đầu vào của generator là một tập nhiễu ngẫu nhiên (noise) và học cách biến đổi bộ nhiễu này thành một mẫu dữ liệu.

Mạng phân biệt, hay discriminator là phần sau của GAN, có tác dụng phân biệt giữa dữ liệu thật có trong tập huấn luyện và dữ liệu được tạo ra từ nhiễu. Nói cách khác, nó cố gắng phân biệt một dữ liệu là đến từ tập huấn luyện gốc hay đến từ mô hình sinh.

Quá trình huấn luyện GAN là quá trình huấn luyện đồng thời cả hai mạng generator và discriminator theo quy tắc ‘Minimax’ giữa generator và discriminator: Mục tiêu là để discriminator phân biệt tốt nhất giữa dữ liệu thật và dữ liệu giả do generator tạo ra, trong khi generator cố gắng đánh lừa discriminator bằng cách tạo ra dữ liệu một cách giống với dữ liệu thật nhất có thể. Quá trình huấn luyện này tối thiểu hoá hàm mất mát đặc biệt, được đề cập trong [13] và có tên là min-max loss:

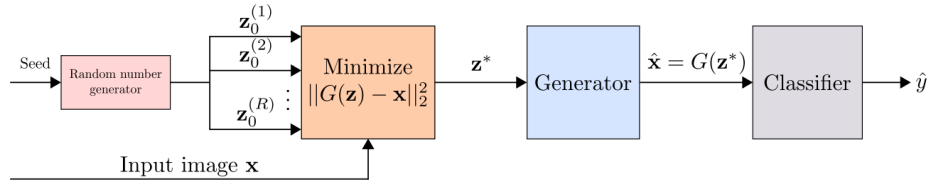


Figure 1: Overview of the Defense-GAN algorithm.

Hình 2: Tổng quan về mô hình GAN

Trong đó:

- p_{data} : Phân phối của dữ liệu thật
- p_z : Phân phối của nhiễu đầu vào
- $D(x)$: Là xác suất mà discriminator dự đoán x là mẫu thật
- $G(z)$: Là dữ liệu giả được tạo ra từ nhiễu z qua G

Trong quá trình huấn luyện, liên tục cập nhật D và G với mục tiêu được nêu ở trên một cách tuần tự. Hàm min-max loss đạt được nghiệm tối ưu khi phân phối xác suất của kết quả đầu ra generator bằng với phân bố dữ liệu thật: $p_{generator} = p_{data}$. Tuy nhiên thông qua thực nghiệm trong [13] cho thấy rằng việc huấn luyện mô hình này không phải đơn giản vì tính lặp lại và phức tạp trong quá trình tính toán của hàm min-max loss. Arjovsky tại [2] đã giới thiệu về Wasserstein GANs (WGANs) là một biến thể của GAN sử dụng khoảng cách Wasserstein để biến đổi hàm min-max loss của GAN về dạng đơn giản hơn về mặt tính toán và hội tụ.

Defense-GAN [13] sử dụng WGAN được huấn luyện trên dữ liệu sạch để khử nhiễu với các mẫu đối kháng. Tại quá trình suy diễn, hình ảnh x cần được phân loại, đầu tiên xác định vectơ ngẫu nhiên z^* để đưa vào bộ phân loại G phù hợp với x . Đưa z^* vào G để sinh ra mẫu $\hat{x} = G(z^*)$. Sử dụng mẫu này để làm dữ liệu đầu vào cho bộ phân loại. Việc thực hiện tính toán z^* được tính xấp xỉ bằng thuật toán Gradient Descent với số bước L cố định.

Khi thực hiện áp dụng trong [2], Defense-GAN được triển khai để bảo vệ mô hình phân loại trước các cuộc tấn công dạng hộp trắng và hộp đen trên nhiều bộ dữ liệu khác nhau về hình ảnh. Kết quả cho ra được so sánh với bộ phòng thủ Magnet cho thấy rằng Defense-GAN đã có khả năng chống lại được các cuộc tấn công khác nhau hiệu quả hơn rõ rệt so với Magnet, độ dự đoán chính xác của mô hình phân loại mục tiêu tăng đáng kể.

Defense-GAN là một bộ phòng thủ đạt hiệu suất tốt và mang tính linh hoạt cao khi có thể triển khai phòng thủ trước nhiều dạng tấn công khác nhau, điều mà các phương pháp phòng thủ khác còn bị hạn chế. Tuy nhiên, để có thể xây dựng được một bộ phòng thủ Defense-GAN đủ tốt không phải là điều đơn giản. Có thể thấy, quá trình huấn luyện GAN sử dụng Gradient Descent để tìm ra cực trị tối ưu gây mất thời gian cũng như sự bất ổn định cho mô hình. Hơn nữa, việc phải lựa chọn siêu tham số cho phù hợp cũng là một bài toán cần phải nghiên cứu thêm trong tương lai để GAN có thể sinh ra mẫu sát với mẫu sạch nhất. Do thời gian suy diễn và lọc nhiễu đối kháng có thể là một rào cản để ứng dụng vào Defense-GAN cho các hệ thống thời gian thực, nơi mà sự kịp thời và chính xác phải được đặt lên hàng đầu. Có thể bộ phòng thủ này thích hợp trang bị cho các hệ thống phân loại tĩnh, hoặc sử dụng kết quả thực nghiệm của bộ phòng thủ này để đối chiếu với các bộ phòng thủ khác trong thí nghiệm liên quan đến tấn công và bảo vệ mô hình phân loại, học sâu.

Trong nghiên cứu này, các tác giả tập trung nghiên cứu phương pháp tạo ra các mẫu sạch từ các mẫu đối kháng. Các mẫu sau khi được thanh lọc sẽ được đưa vào bộ phân loại mục tiêu giúp kết quả không bị ảnh hưởng bởi các cuộc tấn công. Để khắc phục những hạn chế của các phương pháp trước đó như MagNet và Defense-GAN [13], nghiên cứu này đề xuất một phương pháp mới có tên là Purifying Variational AutoEncoder (PuVAE), dựa trên Variational Autoencoder (VAE) [8] để làm sạch các mẫu đối kháng. PuVAE phù hợp với những hệ thống cần phản hồi nhanh như các ứng dụng thời gian thực do nó sử dụng suy luận biến phân tạo ra các mẫu sạch chỉ trong một bước truyền tới.

Tóm lại, đóng góp của nghiên cứu này bao gồm:

1. Đề xuất PuVAE: một phương pháp làm sạch mẫu đối kháng dựa trên VAE.
2. Tăng tốc độ xử lý: PuVAE rút ngắn thời gian tạo ra mẫu đã thanh lọc, nhanh hơn so với các phương pháp phòng thủ hiện tại.
3. Tăng khả năng phòng thủ: Kết quả thử nghiệm cho thấy PuVAE hoạt động ổn định và hiệu quả trên nhiều phương pháp tấn công đối kháng và các tập dữ liệu khác nhau.

3 Tổng quan

3.1 Variational Autoencoder (VAE)

Variational Autoencoder (VAE) là một mô hình sinh (generative model) được giới thiệu lần đầu bởi Diederik P. Kingma và Max Welling [8]. VAE là sự kết hợp giữa mạng nơ-ron sâu và suy luận biến phân (variational inference), cho phép mô hình học phân phối xác suất của dữ liệu phức tạp và từ đó sinh ra các mẫu dữ liệu mới có đặc trưng giống như dữ liệu gốc.

VAE bao gồm hai thành phần chính:

- **Encoder:** Mã hóa dữ liệu đầu vào thành một không gian ẩn (latent space). Thay vì ánh xạ dữ liệu đầu vào thành một điểm cố định, encoder ánh xạ nó thành một phân phối Gaussian đa biến với các tham số trung bình (mean) và phương sai (variance).
- **Decoder:** Từ không gian ẩn, decoder sinh ra dữ liệu mới, nhằm tái tạo lại dữ liệu đầu vào.

Mục tiêu của VAE là tối đa hóa logarit biên độ (marginal log likelihood) của dữ liệu $\log(p(x))$. Do dữ liệu có thể rất phức tạp rất khó để tính trực tiếp $p(x)$, mà thay vào đó các kỹ thuật suy luận biến phân (variational inference) sẽ được dùng để xấp xỉ $p(x)$ thông qua không gian ẩn z . Hàm mục tiêu có công thức là:

$$\log(p(x)) = D_{KL}(q_\phi(z|x) \parallel p_\theta(z|x)) + \mathcal{L}(\theta, \phi, x)$$

Trong đó:

- $D_{KL}(q_\phi(z|x) \parallel p_\theta(z|x))$: là phân kỳ Kullback-Leibler (KL Divergence) giữa phân phối suy luận $q_\phi(z|x)$ và phân phối thực $p_\theta(z|x)$.
- $\mathcal{L}(\theta, \phi, x)$: là biên dưới biến phân (Variational Lower Bound) của $\log(p(x))$.

$$\mathcal{L}(\theta, \phi, x) = -D_{KL}(q_\phi(z|x) \parallel p_\theta(z)) + \mathbb{E}_{z \sim q_\phi(z|x)} [\log(p_\theta(x|z))]$$

Conditional Variational Autoencoder (cVAE) [4] là một mở rộng của VAE. Thay vì chỉ học phân phối tổng quát của dữ liệu, cVAE còn kết hợp thông tin điều kiện (chẳng hạn nhãn của dữ liệu) vào quá trình mã hóa và giải mã.

Trong cVAE:

- **Encoder** và **Decoder** nhận thêm thông tin điều kiện y (nhãn lớp hoặc đặc trưng phụ), giúp mô hình tái tạo dữ liệu tương ứng với từng lớp hoặc đặc trưng cụ thể.
- Hàm mục tiêu của cVAE mở rộng từ VAE với thêm biến điều kiện:

$$\mathcal{L}(\theta, \phi, x, y) = -D_{KL}(q_\phi(z|x, y) \parallel p(z|y)) + \mathbb{E}_{z \sim q_\phi(z|x, y)} [\log(p_\theta(x|z, y))]$$

Nghiên cứu này sử dụng cVAE để học phân phối của từng lớp, hỗ trợ mô hình thanh lọc các mẫu đối kháng bằng cách tái tạo lại các mẫu hợp lệ từ không gian ẩn dựa trên điều kiện lớp, giúp loại bỏ nhiều đối kháng.

3.2 Adversarial Example

Các mạng nơ-ron nhân tạo, hay các mạng học sâu hiện nay đều phải sử dụng một lượng tương lớn đối dữ liệu dựa trên kiến trúc cũng như yêu cầu về độ phức tạp và tính chính xác của mỗi tác vụ được ứng dụng. Vì thế, dữ liệu đầu vào được coi là một trong những thành tố quyết định đến sự thành công của một mô hình học máy nói chung cũng như bất kỳ mạng học sâu nào. Công việc kiểm định và xử lý dữ liệu từ lâu là một thách thức lớn đối với quá trình triển khai hệ thống học máy. Trong một vài trường hợp, dữ liệu đầu vào có thể bị thay đổi một cách tinh vi để gây ảnh hưởng xấu đến mô hình.

Cụ thể, trong bài toán phân loại hình ảnh, để giảm hiệu suất phân loại chính xác của mô hình, hình ảnh trước khi được đưa vào mô hình phân loại có thể được thêm vào một lượng nhiễu mà sao cho hình ảnh được thêm nhiễu khó bị phát hiện bằng mắt thường nhưng gây ra sự thiếu chính xác trong quá trình dự đoán nhãn tương ứng cho ảnh được thêm nhiễu đó. Các chiến lược, thuật toán để tạo ra nhiễu tương ứng được gọi là các cuộc tấn công đối kháng (Adversarial attacks). Những nhiễu được thêm vào hình ảnh sạch trong quá trình tấn công được gọi là nhiễu đối kháng (Adversarial perturbations). Những hình ảnh sau khi được thêm nhiễu đối kháng gây ra sự phân loại sai cho mô hình phân loại mục tiêu là các mẫu đối kháng (Adversarial examples). Việc tấn công sử dụng mẫu đối kháng dựa vào tính chất nhạy cảm với dữ liệu của các mô hình học máy bởi kiến trúc tuyến tính tương ứng [6].

Có nhiều phương pháp tấn công để có thể sinh ra các mẫu đối kháng, nhưng trong phạm vi này sẽ giới thiệu về bốn phương pháp tấn công tận dụng tính chất tuyến tính của mô hình và nhạy cảm với các tính toán gradient để sinh ra mẫu đối kháng cho mô hình phân loại. Tổng quan, mọi cuộc tấn công được xem xét đều đặt mục đích là tìm được nhiễu δ cần được thêm vào đầu vào sạch $x \in \mathbb{R}^n$ để tạo ra mẫu đối kháng. Độ lớn của nhiễu được thêm vào được định nghĩa là ϵ :

$$\epsilon = \|\delta\|_{\infty} = \max_i |\delta_i|$$

Để tạo ra nhiễu khó phát hiện, lí tưởng là ϵ phải đủ nhỏ và hợp lí để nhiễu đối kháng có thể vượt qua được quan sát thông thường đồng thời giúp mẫu đối kháng đủ tác động đến mô hình [13].

3.2.1 Fast Gradient Sign Method (FGSM)

Các mạng nơ-ron sâu dựa vào việc tính toán gradient để điều chỉnh trọng số trong quá trình huấn luyện nhằm tối thiểu hàm mất mát. Gradient tương tự như đạo hàm của hàm số, mô tả độ dốc của hàm mất mát theo đầu vào hoặc các tham số của mô hình. Phương pháp FGSM được giới thiệu tại [6]. FGSM tạo ra mẫu đối kháng bằng cách thêm nhiễu đối kháng nhỏ δ phụ thuộc vào dấu của gradient. Công thức tổng quát của FGSM là:

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

Trong đó:

- x : Đầu vào của mô hình tấn công, hay ảnh sạch.

- x' : Mẫu đối kháng
- ϵ : Hệ số điều chỉnh cường độ nhiễu.
- $J(\theta, x, y)$: Hàm mất mát của mô hình tấn công sử dụng mạng nơ-ron tương ứng với đầu vào x có nhãn là y , các tham số θ của mô hình.
- $\nabla_x J(\theta, x, y)$: Gradient của hàm mất mát theo đầu vào x .

Sử dụng dấu của Gradient để tạo ra nhiễu giúp FGSM có thể tạo ra các mẫu đối kháng không chỉ đơn giản mà còn tối ưu về ý tưởng là tăng giá trị hàm mất mát. Có thể thấy, việc tạo ra mẫu chỉ cần thực hiện trong một bước tính toán gradient duy nhất, nên FGSM được coi là dạng tấn công one-step gradient.

FGSM được thiết kế để tối ưu về mặt tốc độ hơn là việc tối ưu về khả năng tấn công. Về thực nghiệm như ở [6], phương pháp FGSM gây giảm tỉ lệ chính xác xuống mức đáng kể lên những mạng nơ-ron học sâu đơn giản, điều này chứng tỏ FGSM có thể coi như một phương pháp tạo nhiễu đối kháng đơn giản, dễ triển khai cho các cuộc tấn công đối kháng lên mạng phân loại, chủ yếu là phân loại ảnh.

Mặc dù hiệu quả cho ra tốt trong khi việc thiết kế lại đơn giản, FGSM vẫn có thể bị phát hiện và xử lý bởi một số kỹ thuật nhất định liên quan đến việc huấn luyện có điều chỉnh của mô hình, hoặc sự thiếu hiệu quả khi tấn công vào các mạng có độ phức tạp lớn hơn. Điều này yêu cầu sự phát triển về mặt cấu trúc của thuật toán cũng như cách tạo mẫu đối kháng cho các phiên bản phát triển từ FGSM.

3.2.2 Iterative Fast Gradient Sign Method (iFGSM)

Được Kurakin giới thiệu tại [10], iFGSM là một phiên bản mở rộng của thuật toán FGSM theo hướng phức tạp hơn việc thêm nhiễu đối kháng. Thay vì chỉ sử dụng kết quả của một bước gradient duy nhất để sinh ra nhiễu, iFGSM thực hiện nhiều bước lặp kèm với đó thêm vào các tham số tùy chỉnh để sinh ra nhiễu mạnh hơn, làm cho sự sai lệch trong phân loại của mô hình trở nên lớn hơn. Cụ thể, khi áp dụng FGSM theo số lần nhất định, mỗi lần cách nhau một khoảng gọi là step-size, đồng thời nhiễu đối kháng phải nằm trong biên độ tối đa ϵ :

$$x'_{t+1} = \text{Clip}_{x, \epsilon} \{x'_t + \alpha \cdot \text{sign}(\nabla_{x_t} J(\theta, x_t, y))\}$$

Trong đó:

- x_t : Mẫu đối kháng tại lần lặp thứ t .
- α : Hệ số học (learning rate) sau mỗi lần lặp, tương tự như trong Gradient Descent.
- Hàm $\text{Clip}()$ được thể hiện trong công thức trên được dùng để giới hạn sự thay đổi của nhiễu δ không thể vượt quá biên độ ϵ .

Quy trình thực hiện tìm mẫu đối kháng tối ưu theo thuật toán iFGSM:

1. Khởi tạo đầu vào đối kháng $x'_0 = x$.

2. Ở mỗi bước t tiếp theo, cập nhật x_t theo công thức mô tả như trên áp dụng tại giá trị x_{t-1} .
3. Lặp lại (2) cho đến khi đạt được số lần định trước hoặc khi nhiều đạt đến biên.
4. Cắt giá trị x'_t để đảm bảo đầu vào đối kháng nằm trong khoảng giá trị hợp lý.

iFGSM trong thực nghiệm thường tạo ra các mẫu đối kháng hiệu quả hơn so với FGSM với việc triển khai các bước như trên. Biến thể này FGSM có thể giúp bộ tấn công vượt qua được các hệ thống phòng thủ thông thường được dùng để chống lại FGSM. Tuy nhiên, việc tạo ra các mẫu đối kháng hiệu quả từ iFGSM cần phải được thực hiện kỹ lưỡng, vì nó phụ thuộc vào sự điều chỉnh của các siêu tham số xuất hiện trong mô hình.

3.2.3 Randomized Fast Gradient Sign Method (RAND+FGSM)

Với các mô hình đã được huấn luyện bằng một số phương pháp nhất định để có khả năng phân loại được các bộ dữ liệu chứa nhiều đối kháng, thường là được tạo bởi FGSM thì sẽ rất khó khăn để có thể tấn công các mạng kiểu này. Tramèr đã nêu ra một biến thể phát triển khác ngoài iFGSM tại [14] dựa trên sự ngẫu nhiên của phân phối chuẩn làm cho việc phòng thủ trở nên khó khăn hơn bởi tính bất định của thông tin được gọi là Randomized FGSM (RAND+FGSM). Quy trình để tạo ra mẫu đối nghịch được như sau:

$$x' = x + \alpha \cdot \text{sign}(N(0^n, I^n))$$

$$\tilde{x} = x + (\epsilon - \alpha) \cdot \text{sign} \nabla_{x'} J(\theta, x', y)$$

Trong đó:

- $N(0^n, I^n)$: Phân phối chuẩn $(0,1)$.
- α : Hệ số điều chỉnh biên độ của nhiễu, $\alpha < \epsilon$.

Thông qua bước thêm một nhiễu ngẫu nhiên vào mẫu đầu vào ban đầu, việc tính toán gradient bây giờ sẽ bớt phụ thuộc vào mẫu ban đầu hơn, giúp cho bộ tấn công này trở nên linh hoạt hơn khi sự biến đổi bây giờ dựa trên lân cận của x thay vì chính nó.

Phương pháp này giúp tạo ra nhiều mẫu đối kháng khác nhau ngay cả khi sử dụng cùng một đầu vào, từ đó các bộ phân loại dù được huấn luyện đối kháng cơ bản nhưng vẫn có thể phân loại sai. Tuy nhiên tương tự iFGSM, bộ tấn công yêu cầu nhiều tài nguyên tính toán hơn FGSM thông thường, đồng thời độ hiệu quả cũng phụ thuộc vào siêu tham số α, ϵ .

3.2.4 Carlini-Wagner attack (CW attack)

Phương pháp này được Carlini và Wagner giới thiệu tại [3]. Đây được coi là phương pháp tấn công hiệu quả nhất trong thời điểm bấy giờ, cuộc tấn công này có thể giảm khả năng chính xác của mô hình phân lớp về gần mức 0%.

Mục tiêu của CW attack là tạo ra mẫu có nhiễu đối kháng δ nhỏ sao cho mẫu đối kháng $\tilde{x} = x + \delta$ gây sai lệch trong dự đoán của mô hình nhưng vẫn giữ lại sự tương đồng cao

với x . Nhiều này có thiết kế đủ nhỏ để không thể phát hiện bằng mắt thường. Cụ thể, giá trị δ^* được chọn bằng bài toán tối ưu của CW attack:

$$\begin{aligned} \min_{\delta} \quad & \|\delta\|_p + c \cdot f(x + \delta) \\ \text{s.t.} \quad & x + \delta \in [0, 1]^n \end{aligned}$$

Trong đó:

- $\|\delta\|_p$: norm- p của nhiễu, dùng để đo độ lớn của δ . Thông thường xét l_2, l_0, l_∞ .
- c : Trọng số của lớp phân loại.
- $f(x + \delta)$: Hàm phân loại mục tiêu dùng để phân loại.

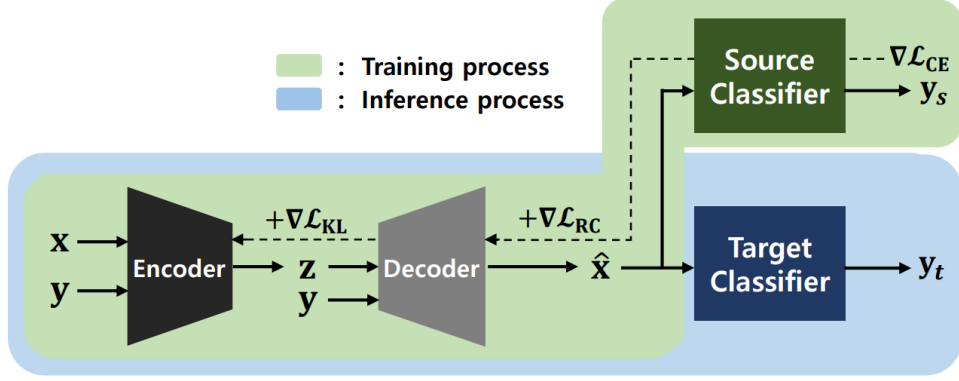
Qua nhiều thực nghiệm về việc tấn công các mạng phân loại sử dụng CW attack cho thấy phương pháp này thể hiện hiệu quả vượt trội so với các phương pháp từng được đề xuất trước đó. Phương pháp được áp dụng trên nhiều bộ dữ liệu và phương pháp tấn công khác nhau (tấn công hộp đen, hộp trắng) đều có sự giảm dần rõ rệt về tính chính xác của mô hình so với bộ dữ liệu sạch.

CW attack được sử dụng rộng rãi trong nghiên cứu và đánh giá tính bền vững của các mô hình học sâu trước sự tổn thương dữ liệu. Tác giả lựa chọn $p = 2$ trong phạm vi nghiên cứu này.

4 Phương pháp được đề xuất

PuVAE là phương pháp phòng thủ mới được đề xuất nhằm thanh lọc nhiễu đối kháng (adversarial noise) khỏi dữ liệu đầu vào dựa trên nền tảng của mạng VAE. Nhiễu đối kháng là những nhiễu nhỏ được thêm vào một cách tinh vi đánh lừa hệ thống học sâu, khiến mô hình đưa ra các dự đoán sai. Mục tiêu chính của PuVAE là loại bỏ các nhiễu này để dữ liệu đầu vào trở nên "sạch" hơn trước khi được đưa vào mô hình phân loại. Tập dữ liệu $\mathcal{X}_{data}$ bao gồm các mẫu dữ liệu $\mathbf{x}_{data} \in \mathbb{R}^d$, với d là số chiều của không gian dữ liệu. Nhãn các lớp tương ứng với mỗi mẫu dữ liệu được ký hiệu là $\mathbf{y}_{data} \in \mathbb{R}^c$ (one-hot vector), trong đó c là số lớp.

Bộ phân loại mục tiêu $\mathcal{M}_t$ là mô hình mà kẻ tấn công muốn nhắm đến. Tập $\mathcal{X}_{adv}$ chứa các mẫu đối kháng $x_{adv} \in \mathbb{R}^d$, đây là các mẫu đã bị thay đổi bằng cách thêm nhiễu đối kháng, nhằm đánh lừa mô hình mục tiêu. Tập hợp X bao gồm cả mẫu sạch và mẫu đối kháng. Tập này được sử dụng trong quá trình suy luận để kiểm tra hiệu quả của phương pháp phòng thủ. Mỗi mẫu x từ tập X có thể là một mẫu dữ liệu bình thường hoặc một mẫu đối kháng, và PuVAE sẽ thực hiện quá trình thanh lọc trên các mẫu này để loại bỏ nhiễu. Hình 3 khái quát của mô hình được đề xuất.



Hình 3: Tổng quan phương pháp được đề xuất.

4.1 Quá trình huấn luyện PuVAE

PuVAE gồm hai thành phần chính là encoder(mã hóa) và decoder(giải mã), tương tự với cấu trúc của các mô hình VAE thông thường.

Encoder nhận đầu vào là một cặp dữ liệu và nhãn lớp (x_{data}, y_{data}) để mã hóa dữ liệu cùng ngữ cảnh theo từng lớp. Đầu ra của encoder bao gồm μ (trung bình) và σ (độ lệch chuẩn) của một phân phối Gaussian trên không gian ẩn.

$$\mu, \sigma = \text{Encoder}(x_{data}, y_{data})$$

Thay vì chỉ ánh xạ đầu vào thành một điểm cố định trong không gian ẩn, PuVAE sử dụng công thức:

$$z = \mu + \epsilon \cdot \sigma$$

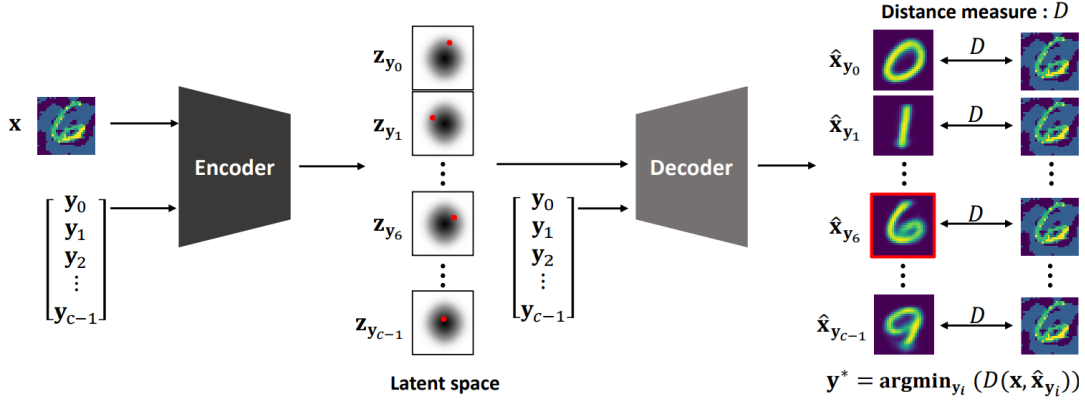
$$\epsilon \sim N(0, \sigma_\epsilon \mathbf{I})$$

σ_ϵ là một hyperparameter nhằm điều chỉnh độ rộng của không gian ẩn (trong các thử nghiệm, tác giả đặt $\sigma_\epsilon = 1$).

Ngoài ra, PuVAE còn sử dụng mạng dilated convolution (tích chập giãn nở) trong encoder. Trong các bài toán phân loại thông thường, các mạng tích chập thường sử dụng các phép pooling và stride để giảm số chiều, tuy nhiên điều này có thể làm mất đi các chi tiết nhỏ trong dữ liệu. Đối với nhiệm vụ thanh lọc nhiễu đối kháng, các chi tiết nhỏ lại rất quan trọng, vì chúng giúp phân biệt giữa dữ liệu gốc và dữ liệu bị nhiễu. Mạng dilated convolution giúp mở rộng trường nhìn mà không làm mất mát thông tin, tạo ra một biểu diễn chi tiết hơn cho mỗi mẫu dữ liệu đầu vào.

Decoder trong PuVAE đảm nhận nhiệm vụ tái tạo lại dữ liệu từ không gian ẩn. Với đầu vào là vector z và nhãn y_{data} , decoder sẽ học cách chuyển đổi vector này trở lại không gian dữ liệu gốc. Việc cung cấp nhãn lớp trong quá trình decoder giúp PuVAE tái tạo các mẫu dựa trên cấu trúc và đặc trưng của từng lớp, làm cho quá trình tái tạo mẫu trở nên cụ thể và có ngữ cảnh hơn. Kết quả là $\hat{x}$ được tạo ra có cùng số chiều với input x_{data} , được tái tạo từ không gian ẩn, đã được loại bỏ các nhiễu đối kháng.

$$\hat{x} = \text{Decoder}(z, y_{data})$$



Hình 4: Quá trình suy luận của PuVAE với ảnh MNIST.

Trong quá trình training, tương tự như cVAE, PuVAE cũng cố gắng maximize lower bound. Mục tiêu là cần cực tiểu hóa các hàm mất mát.

1. Reconstruction loss (Hàm mất mát tái tạo) L_{RC} giúp đảm bảo mẫu tái tạo có đặc trưng giống với dữ liệu ban đầu. Công thức L_{RC} có dạng:

$$L_{RC} = \mathbf{x}_{data} \log \hat{\mathbf{x}} + (1 - \mathbf{x}_{data}) \log(1 - \hat{\mathbf{x}})$$

2. Hàm mất mát Kullback-Leibler divergence L_{KL} nhằm đo lường sự khác biệt giữa phân phối trong không gian ẩn mà encoder học được và phân phối chuẩn tắc, giúp duy trì tính ổn định và không bị nhiễu bởi các yếu tố ngẫu nhiên không mong muốn.

$$L_{KL} = \mu^2 + \sigma^2 - \log(\sigma^2) - 1$$

3. Hàm mất mát Cross-Entropy L_{CE} đảm bảo các mẫu tái tạo được gán nhãn đúng lớp.

$$L_{CE} = \mathbf{y}_{data} \log y_s + (1 - \mathbf{y}_{data}) \log(1 - y_s)$$

Trong đó, y_s là đầu ra của *source classifier* M_s . Đây là bước bổ sung so với các VAE thông thường, giúp PuVAE trở nên đặc biệt hiệu quả trong việc giữ nguyên đặc trưng lớp, ngay cả khi có nhiễu đối kháng.

Quá trình huấn luyện của PuVAE sử dụng phương pháp tối ưu hóa stochastic gradient descent, trong đó ba thành phần mất mát L_{RC}, L_{KL}, L_{CE} được kết hợp với nhau.

$$\nabla(\lambda_{RC}L_{RC} + \lambda_{KL}L_{KL} + \lambda_{CE}L_{CE})$$

Trong đó λ_{RC} , λ_{KL} , và λ_{CE} là các hệ số điều chỉnh trọng số của từng thành phần.

4.2 Quá trình sinh mẫu đã được thanh lọc

Trong PuVAE, sau khi mô hình đã được huấn luyện với các thành phần mất mát khác nhau, bước tiếp theo là sử dụng nó để xử lý các mẫu đầu vào trong giai đoạn inference (suy luận). Mục tiêu là chuyển các mẫu bị nhiễu đối kháng trở về dạng "sạch" và phù hợp với đặc trưng của lớp ban đầu.

Đầu tiên, khi có một đầu vào, PuVAE được sử dụng để tạo ra các đại diện cho mẫu đó trong không gian ẩn tương ứng với từng lớp. Đối với mỗi lớp y_i trong tập hợp các lớp C , PuVAE sẽ dự đoán một phiên bản của mẫu đầu vào x dựa trên nhãn lớp y_i . Công thức sử dụng trong bước này là:

$$\hat{\mathbf{x}}_{y_i} = \text{PuVAE}(\mathbf{x}, y_i)$$

Trong đó:

- $\mathbf{x}$: mẫu đầu vào, có thể chứa nhiều đối kháng.
- y_i : nhãn lớp i trong tập lớp C .
- $\hat{\mathbf{x}}_{y_i}$: phiên bản của mẫu $\mathbf{x}$ được tái tạo trong không gian ẩn của lớp y_i .

Tiếp theo là xác định phiên bản nào của mẫu đầu vào gần nhất với đặc trưng gốc của nó. Khoảng cách giữa mẫu đầu vào $\mathbf{x}$ và từng đại diện $\hat{\mathbf{x}}_{y_i}$ được đo bằng hàm D , như hình 4. Công thức được sử dụng là:

$$y^* = \arg \min_{y_i \in C} D(\mathbf{x}, \hat{\mathbf{x}}_{y_i})$$

trong đó:

- D : là một hàm đo lường khoảng cách, nghiên cứu này sử dụng Root Mean Square Error - RMSE (căn bậc hai trung bình bình phương để đo khoảng cách).
- y^* : là lớp có khoảng cách nhỏ nhất.

Khi đã xác định được lớp y^* , PuVAE sẽ lấy đại diện của lớp đó làm *mẫu thanh lọc*. Mẫu thanh lọc này được ký hiệu là $\mathbf{x}_{\text{purified}}$:

$$\mathbf{x}_{\text{purified}} = \hat{\mathbf{x}}_{y^*}$$

Bằng cách dự đoán đại diện của mẫu đầu vào cho từng lớp, PuVAE tạo ra nhiều phiên bản tiềm năng của mẫu đầu vào mà không có nhiễu đối kháng. Mỗi phiên bản này sẽ nằm trên một manifold (mặt nhiều chiều) trong không gian ẩn tương ứng với lớp đó. Cuối cùng, mẫu thanh lọc $\mathbf{x}_{\text{purified}}$ được đưa vào target classifier (bộ phân loại mục tiêu) M_t để xác định lớp của nó.

$$y_t = M_t(\mathbf{x}_{\text{purified}})$$

trong đó:

- M_t : là bộ phân loại mục tiêu ban đầu mà kẻ tấn công đã cố gắng đánh lừa.
- y_t : là nhãn lớp dự đoán của mẫu thanh lọc.

Quá trình PuVAE đảm bảo rằng mẫu đầu vào đã được làm sạch trước khi được phân loại, làm giảm khả năng bộ phân loại bị đánh lừa bởi nhiễu đối kháng.

5 Thực nghiệm

Trong phần này, nhóm tác giả xác định thiết lập tối ưu cho PuVAE và trình bày hiệu suất phòng thủ của PuVAE trước các cuộc tấn công đối nghịch đã được đề cập trước đó. Về công cụ thử nghiệm, nhóm tác giả đã sử dụng Tensorflow (1.12.0). Về phần cấu hình máy, GPU, NVIDIA TITAN V (12 GB) và CPU, Intel Xeon E5-2690 v4 (2,6 GHz) đã được sử dụng. Về tập dữ liệu, nhóm tác giả sử dụng 3 tập dữ liệu: MNIST [11] là một tập dữ liệu chữ số viết tay, Fashion-MNIST [15] là một tập dữ liệu hình ảnh đối tượng quần áo và CIFAR-10 [9] là một tập dữ liệu hình ảnh tự nhiên. Mỗi tập dữ liệu gồm 50000 ảnh để đào tạo và 10000 ảnh để kiểm tra. Dữ liệu được chuẩn hóa trong khoảng từ 0 đến 1. Các mô hình FGSM, iFGSM, RAND+FGSM và CW được sử dụng cho các lần thử nghiệm. Trong đó, FGSM, iFGSM và RAND+FGSM được tạo ra với kích thước nhiễu loạn đối nghịch (adversarial perturbation size) là 0,3 cho các tập dữ liệu MNIST và Fashion-MNIST và 0,06 cho tập dữ liệu CIFAR-10. Đồng thời, nhóm tác giả đặt giá trị giới hạn trên cho nhiễu ngẫu nhiên của RAND+FGSM là 0,05 cho các tập dữ liệu MNIST và FashionMNIST và 0,005 cho tập dữ liệu CIFAR-10. Số lần lặp của các cuộc tấn công bằng CW được đặt là 100 trên tất cả các tập dữ liệu. Cuối cùng, hiệu suất của các cơ chế phòng thủ được đo bằng độ chính xác (accuracy) của bộ phân loại mục tiêu.

Các bộ mã hóa và bộ giải mã của PuVAE là các mạng nơ-ron và có kiến trúc được trình bày trong hình 7. Trong đó:

- Dilated $Conv(n, k \times k, r)$ là một lớp tích chập giãn nở với n bản đồ đặc trưng, kích thước bộ lọc là $k \times k$ và tốc độ giãn nở là r .
- $Deconv(n, k \times k, s)$ là một lớp giải tích chập với n bản đồ đặc trưng, kích thước bộ lọc là $k \times k$, và bước tiến (stride) s .
- $FC(m)$ là một lớp fully connected với m đơn vị đầu ra.
- ReLU là hàm kích hoạt tuyến tính chỉnh lưu.

Trong kiến trúc của bộ mã hóa, nửa đầu của lớp cuối cùng có 32 đơn vị đầu ra được sử dụng để đại diện cho μ , và nửa còn lại được đưa qua hàm softplus để tính σ .

Ngoài ra, nhóm tác giả cũng thử nghiệm trên Defense-GAN và mạng cải cách của MagNet để so sánh, kiến trúc sử dụng được đề xuất trong [13], [12]. Kiến trúc của M_t được trình bày trong hình 6 dưới đây, trong đó kiến trúc A và D cho tập dữ liệu MNIST và Fashion-MNIST, kiến trúc C và D cho tập dữ liệu CIFAR-10.

Encoder	Decoder
Dilated Conv(32, 7×7 , 2)	FC(512)
ReLU	ReLU
Dilated Conv(32, 7×7 , 2)	Deconv(32, 7×7 , 2)
ReLU	ReLU
Dilated Conv(32, 7×7 , 2)	Deconv(32, 7×7 , 2)
ReLU	ReLU
FC(1024)	Deconv(32, 7×7 , 2)
ReLU	Sigmoid
FC(1024)	
ReLU	
FC(64)	
Softplus (for σ only)	

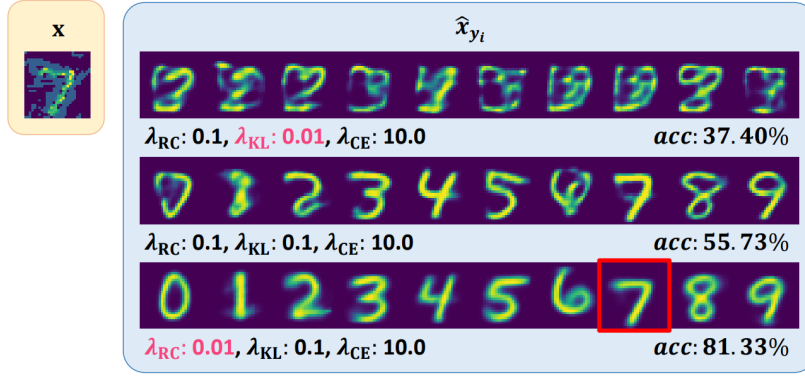
Hình 5: Kiến trúc mạng nơ-ron của PuVAE

A	B	C	D
Conv(64, 5×5 , 1)	Conv(128, 3×3 , 1)	Conv(64, 5×5 , 1)	Conv(128, 3×3 , 1)
ReLU	ReLU	ReLU	ReLU
Conv(64, 5×5 , 2)	Conv(64, 3×3 , 2)	Conv(128, 5×5 , 1)	Conv(256, 3×3 , 1)
ReLU	ReLU	ReLU	ReLU
Dropout(0.25)	Dropout(0.25)	Conv(256, 5×5 , 2)	Conv(512, 3×3 , 2)
FC(128)	FC(128)	ReLU	ReLU
ReLU	ReLU	Dropout(0.25)	Dropout(0.25)
Dropout(0.5)	Dropout(0.5)	FC(512)	FC(512)
FC(10) + Softmax	FC(10) + Softmax	ReLU	ReLU
		Dropout(0.5)	Dropout(0.5)
		FC(256)	FC(10) + Softmax
		ReLU	
		FC(10) + Softmax	

Hình 6: Kiến trúc mạng nơ-ron dùng cho bộ phân loại mục tiêu.

5.1 Ảnh hưởng của hệ số trong quá trình đào tạo PuVAE

Theo thử nghiệm của nhóm tác giả, các hệ số λ_{RC} , λ_{KL} , λ_{CE} có ảnh hưởng lớn đến hiệu suất phòng thủ của mô hình PuVAE. Hình 7 dưới đây minh họa đặc điểm của các mẫu được tạo ra dựa trên sự kết hợp của ba hệ số trên.



Hình 7: So sánh các hệ số trong quá trình training PuVAE.

Khi λ_{RC} lớn hơn λ_{KL} , ràng buộc về phân phối hậu nghiệm của bộ mã hóa giảm đi khiến cho bộ mã hóa dễ dàng ánh xạ các mẫu đầu vào vào vùng có khả năng thấp của không gian tiềm ẩn. Đặc điểm này làm cho bộ giải mã tạo ra hình ảnh khác thường như được minh họa ở hàng đầu tiên của hình 7. Khi λ_{KL} lớn, đặc trưng của mỗi lớp được tạo ra rõ ràng. Do đó, có thể tạo ra các mẫu thể hiện các đặc điểm riêng biệt của từng lớp ngay cả khi có một mẫu đầu vào ở các lớp khác như được minh họa ở hàng cuối của hình 7.

Ô màu đỏ trong hình 7 cho thấy rằng mẫu đã được làm sạch $x_{purified}$ gần giống với mẫu đầu vào x nhưng không còn nhiễu đối nghịch. Nhóm tác giả đã thử nghiệm và tìm ra được bộ hệ số cho ra hiệu suất phòng thủ tốt nhất như được minh họa ở hàng cuối cùng của hình 7, giá trị của các hệ số λ_{RC} , λ_{KL} và λ_{CE} lần lượt là 0,01, 0,1 và 10.

5.2 Hiệu suất phòng thủ

Nhóm tác giả đã tiến hành thử nghiệm và so sánh hiệu suất phòng thủ của mô hình PuVAE với khả năng phòng thủ đối đa của huấn luyện đối kháng (adversarial training) - kỹ thuật phòng thủ trong đó mô hình được huấn luyện với cả dữ liệu sạch và dữ liệu nhiễu đối kháng, mô hình MagNet và mô hình Defense-GAN. Nhóm tác giả đặt số lần lặp là 200 và số ứng viên là 20. Thử nghiệm trên các tập dữ liệu MNIST, FashionMNIST và CIFAR-10, hiệu suất phòng thủ của các mô hình phòng thủ trên được thể hiện trong các hình 8, 9 và 10 dưới đây.

Classifier	Attacks	No Attack	No Defense	Adv. Tr.	MagNet	Defense-GAN	PuVAE
A	FGSM (0.3)	99.51	12.46	78.57	26.90	85.29	81.33
	iFGSM (0.3)	99.51	0.00	91.72	72.48	87.40	92.33
	RAND+FGSM (0.05, 0.3)	99.51	10.64	84.21	32.34	89.02	82.70
	CW (100)	99.51	0.43	18.80	18.80	90.04	90.80
B	FGSM (0.3)	99.29	28.85	88.49	71.70	86.03	88.25
	iFGSM (0.3)	99.29	0.04	93.56	86.29	88.55	92.25
	RAND+FGSM (0.05, 0.3)	99.29	6.42	88.44	41.66	89.76	85.03
	CW (100)	99.29	0.59	19.20	19.10	90.76	92.92

Hình 8: Hiệu suất phòng thủ trên tập dữ liệu MNIST.

Classifier	Attacks	No Attack	No Defense	Adv. Tr.	MagNet	Defense-GAN	PuVAE
A	FGSM (0.3)	93.46	4.06	11.51	11.71	56.67	59.18
	iFGSM (0.3)	93.46	0.00	50.31	34.70	65.11	72.51
	RAND+FGSM (0.05, 0.3)	93.46	1.60	8.72	6.96	59.52	45.13
	CW (100)	93.46	5.01	18.00	15.50	68.60	80.59
B	FGSM (0.3)	93.54	2.75	8.00	15.30	58.20	52.46
	iFGSM (0.3)	93.54	0.00	46.97	41.84	66.06	71.04
	RAND+FGSM (0.05, 0.3)	93.54	1.34	9.93	7.44	60.98	52.22
	CW (100)	93.54	4.87	16.70	16.70	67.62	79.42

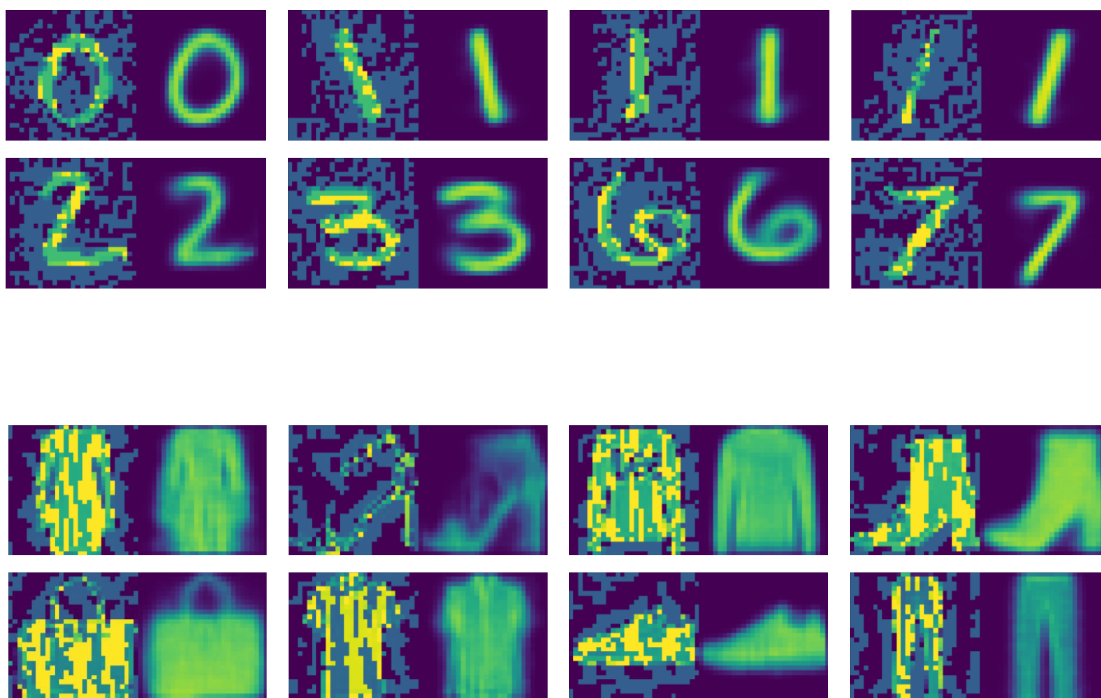
Hình 9: Hiệu suất phòng thủ trên tập dữ liệu Fashion-MNIST.

Classifier	Attacks	No Attack	No Defense	Adv. Tr.	MagNet	Defense-GAN	PuVAE
C	FGSM (0.06)	82.13	3.82	19.07	18.58	29.74	33.71
	iFGSM (0.06)	82.13	0.32	24.13	27.00	33.69	35.49
	RAND+FGSM (0.005, 0.06)	82.13	4.84	21.27	19.99	36.74	33.76
	CW (100)	82.13	9.88	56.48	40.13	38.13	36.36
D	FGSM (0.06)	80.30	3.19	15.00	18.98	30.11	33.20
	iFGSM (0.06)	80.30	0.43	21.51	29.95	32.47	34.48
	RAND+FGSM (0.005, 0.06)	80.30	4.13	18.55	20.44	35.76	34.40
	CW (100)	80.30	9.92	13.64	15.71	28.10	31.70

Hình 10: Hiệu suất phòng thủ trên tập dữ liệu CIFAR10.

Trong hình 8, ta có thể dễ dàng thấy hiệu suất phòng thủ của PuVAE vượt trội hơn MagNet trên tất cả các cuộc tấn công, Defense-GAN có hiệu suất tương đương với PuVAE. Phương pháp huấn luyện đối kháng cũng có hiệu suất tương đương với PuVAE trong các cuộc tấn công của FGSM, iFGSM và RAND+FGSM (vì nhóm tác giả sử dụng các gradient từ M_t để huấn luyện đối kháng nên dễ dàng chống lại các cuộc tấn công dựa trên gradient), đối với các cuộc tấn công CW thì có hiệu suất khá thấp.

Trong hình 9, PuVAE cho thấy hiệu suất phòng thủ tốt nhất trong việc chống lại iFGSM và các cuộc tấn công CW trong cả hai kiến trúc A và B. Mặc dù khi bị iFGSM tấn công mà không có phòng thủ, hiệu suất luôn kém nhất, điều này cho thấy việc chống lại iFGSM là khó khăn nhưng mô hình PuVAE vẫn phòng thủ khá hiệu quả. Các hình ảnh đã được làm sạch trên tập dữ liệu MNIST và Fashion-MNIST được trực quan hóa trong hình 11 dưới đây.



Hình 11: Hiệu suất phòng thủ trên tập dữ liệu Fashion-MNIST.

Trong hình 10, tập dữ liệu CIFAR-10 cho thấy độ chính xác của tất cả các phương pháp phòng thủ đều thấp. Phương pháp huấn luyện đối kháng thể hiện hiệu suất phòng thủ tốt nhất ở một cài đặt nhất định, nhưng điều này cũng cho thấy nó không ổn định mà tùy thuộc vào các mô hình và các cuộc tấn công. PuVAE cho thấy hiệu suất tốt trong nhiều cuộc tấn công với kiến trúc mô hình khác nhau. Điều này chỉ ra rằng phương pháp được đề xuất có khả năng phòng thủ chung trong nhiều cuộc tấn công khác nhau.

5.3 Hiệu suất trong giới hạn thời gian hợp lý

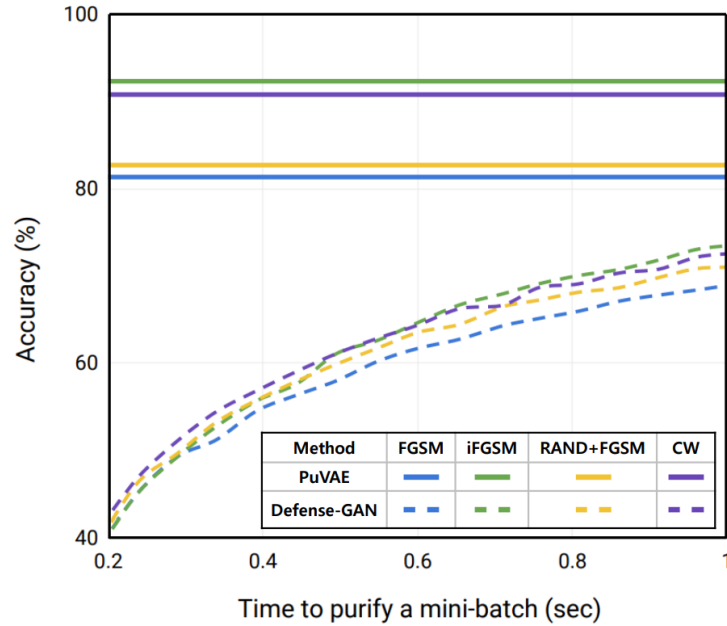
Các phương pháp phòng thủ như Defense-GAN và PuVAE loại bỏ các mẫu đối nghịch theo cách tiền xử lý, tức là các mẫu này được loại bỏ khỏi hình ảnh trước khi đưa vào mô hình phân loại. So với PuVAE, Defense-GAN cần nhiều thời gian hơn để đạt hiệu suất tối đa. Cụ thể, Defense-GAN mất khoảng 14,8 giây để làm sạch một mini-batch gồm 128 hình ảnh MNIST, trong khi đó PuVAE chỉ mất 0,114 giây, cho phép inference nhanh hơn gần 130 lần so với Defense-GAN. Kết quả được thể hiện trong hình 12 dưới đây.

Method	PuVAE	Defense-GAN
Time (seconds)	0.11	14.80
Ratio	1	134.55

Hình 12: So sánh thời gian suy luận của PuVAE và Defense-GAN.

Trong thực tế, có nhiều ứng dụng thời gian thực đã được áp dụng mô hình học sâu, đặc biệt là trong lĩnh vực lái xe tự động. Lĩnh vực này đòi hỏi thời gian xử lý phải nhanh chóng, do đó phương pháp Defense-GAN không hiệu quả trong thực tế mà đòi hỏi phải có phương pháp phòng thủ khác tiết kiệm thời gian hơn. Điều này đã làm nổi bật hơn ưu điểm về thời gian xử lý của PuVAE.

Để chứng minh khả năng xử lý vượt trội về mặt thời gian của PuVAE so với Defense-GAN, nhóm tác giả đã thực hiện đo lường hiệu suất dựa trên giới hạn thời gian hợp lý với tập dữ liệu MNIST. Cụ thể, trong các thử nghiệm, nhóm tác giả đặt giới hạn thời gian là một giây. Kết quả được thể hiện trong hình 13, các đường liền nét thể hiện hiệu suất của PuVAE và các đường nét đứt thể hiện hiệu suất của Defense-GAN. Mỗi màu biểu thị một phương pháp tấn công khác nhau. PuVAE chỉ thực hiện một lần inference, còn Defense-GAN phải tạo ra một vector ẩn theo chu kỳ lặp bằng quy trình tối ưu hóa dựa trên gradient, hiệu suất sẽ tăng lên theo thời gian nhưng không thể đạt được hiệu suất của PuVAE trong giới hạn thời gian. Do đó, PuVAE vượt trội hơn hẳn Defense-GAN trong một giới hạn thời gian và cũng là phương pháp hiệu quả hơn các phương pháp tiên tiến nhất đối với các ứng dụng thời gian thực.



Hình 13: So sánh thời gian suy luận của PuVAE và Defense-GAN.

Điều kiện về ràng buộc thời gian là vô cùng cần thiết để so sánh hai phương pháp PuVAE và Defense-GAN vì Defense-GAN có thời gian inference vượt xa PuVAE. Nhóm tác giả đã thực hiện so sánh hai phương pháp này trong giới hạn thời gian một giây và kết quả được thể hiện như trong hình 14 dưới đây. Từ hình 14 có thể thấy hiệu suất của Defense-GAN thấp hơn đáng kể so với hiệu suất tối đa của nó. Do vậy, PuVAE một lần nữa khẳng định ưu điểm của mình, nó hiệu quả hơn trong các tình huống thực tế vì nó có hiệu suất cao nhất trong điều kiện về giới hạn thời gian.

Method	FGSM	iFGSM	RAND+FGSM	CW
Adv. Tr.	78.57	91.72	84.21	18.80
MagNet	26.90	72.48	32.34	18.80
Defense-GAN	69.25	73.85	71.50	72.79
PuVAE	81.33	92.33	82.70	90.80

Hình 14: Độ chính xác của phương pháp defense trong một giây.

6 Kết luận

PuVAE là một phương pháp phòng thủ mới dựa trên CAE nhằm đối phó với các cuộc tấn công đối kháng trong học sâu. PuVAE không chỉ nâng cao hiệu quả thanh lọc nhiễu đối kháng mà còn khắc phục được những hạn chế của các phương pháp truyền thống. PuVAE được phát triển với mục tiêu:

- Tăng cường khả năng bảo vệ của các mô hình học sâu khỏi các cuộc tấn công đối kháng.
- Giảm thiểu tác động của nhiễu đối kháng trong dữ liệu đầu vào bằng cách sử dụng một quy trình thanh lọc dữ liệu, giúp tái tạo các mẫu đầu vào "sạch" và phù hợp với đặc trưng của lớp gốc.

PuVAE dựa trên cấu trúc của Conditional Variational Autoencoder (cVAE), với các thành phần chính như sau:

- Encoder: Encoder nhận dữ liệu đầu vào và nhúng để tạo ra một không gian ẩn.
- Decoder: Decoder sử dụng thông tin lớp và biểu diễn trên không gian ẩn của dữ liệu đầu vào để tái tạo lại mẫu đầu vào, đồng thời loại bỏ các yếu tố nhiễu đối kháng.

Các thí nghiệm cho thấy:

- Hiệu quả phòng thủ cao: PuVAE đạt được khả năng bảo vệ so sánh được với các phương pháp nổi bật như Defense-GAN.
- Tối ưu về thời gian: PuVAE vượt trội hơn Defense-GAN khi xử lý các mẫu đối kháng trong thời gian giới hạn, giúp tăng hiệu quả xử lý mà không làm giảm khả năng phòng thủ.

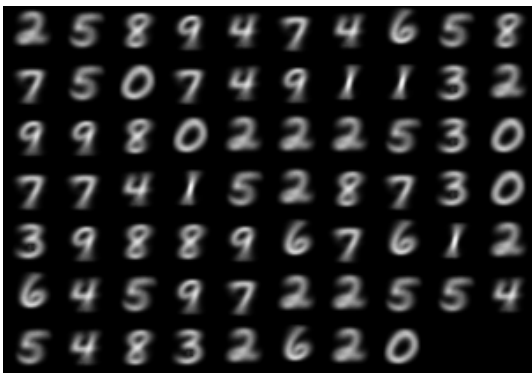
Một vài ứng dụng tiềm năng của PuVAE:

- Xe tự lái: giúp tăng cường độ an toàn cho hệ thống.
- Nhận diện khuôn mặt: Thanh lọc nhiễu đối kháng trong các hệ thống bảo mật bằng nhận diện khuôn mặt.
- Giám sát và an ninh: Bảo vệ các hệ thống giám sát khỏi các cuộc tấn công đối kháng, đảm bảo tính toàn vẹn của dữ liệu.

7 Phụ lục

7.1 Kết quả cài đặt

Kết quả ban đầu của nhóm chúng em khi cài đặt mô hình sát với kiến trúc và bộ tham số được đề xuất giống trong hình 15 như sau. Hình 16 là kết quả mới của mô hình sau khi đã có sự điều chỉnh bộ tham số $\lambda_{KL} = 0.0025$ và $\lambda_{RC} = 1$. Có thể thấy, mặc dù chưa được giống y hệt với ảnh gốc, nhưng mô hình đã có sự cải thiện rõ rệt.

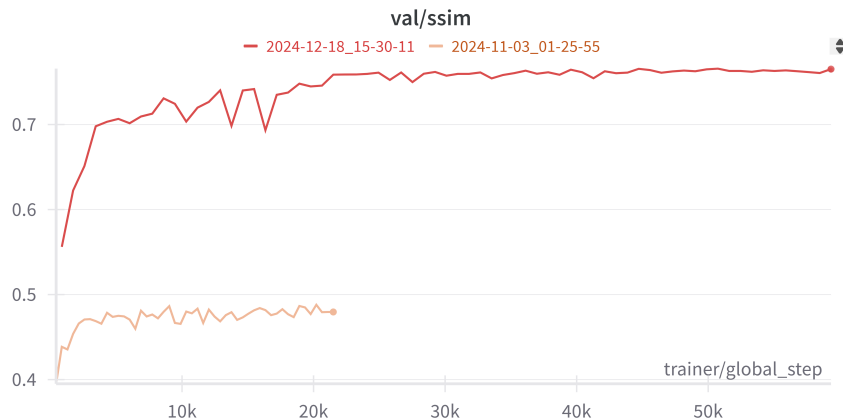


Hình 15: Kết quả cài đặt cũ



Hình 16: Kết quả cài đặt mới

Đường màu đỏ đại diện cho chỉ số SSIM của mô hình mới theo như hình 17 hội tụ chỉ sau khi huấn luyện trên 69 epochs kết hợp với kỹ thuật *Early Stopping*, đạt kết quả khá tốt (0.76).



Hình 17: Chỉ số SSIM

7.2 Hiệu suất của mô hình khi tấn công đối kháng theo phân phối chuẩn

Khi dữ liệu bị tấn công có cùng phân phối với dữ liệu ảnh sạch, PuVAE có thể chỉ loại bỏ được một phần nhiễu.

- Do khi nhiễu được thiết kế để ngẫu nhiên giống như phân phối ảnh sạch khiến PuVAE khó phân biệt nhiễu và đặc trưng thật và có xu hướng coi nhiễu đó là hợp lệ.
- Tuy nhiên PuVAE vẫn có khả năng loại bỏ một phần nhiễu, do nó không dự đoán trên một điểm dữ liệu duy nhất mà các dự đoán sẽ được lấy ngẫu nhiên theo phân phối Gaussian xung quanh điểm đó.

$$\mathbf{z} = \mu + \epsilon \cdot \sigma$$

$$\epsilon \sim N(0, \sigma_\epsilon \mathbf{I})$$

7.3 Hiệu suất của mô hình khi dữ liệu có kích thước lớn



Hình 18: Kết quả của ảnh tái tạo trên bộ dữ liệu STL10

Thực nghiệm trên tập STL10 (64×64) cho thấy ảnh tái tạo lại còn chưa tốt. Nguyên nhân là bởi kiến trúc mô hình VAE mà nhóm tác giả đề xuất còn đơn giản, do vậy, mô hình không đủ khả năng học được cách biểu diễn phân phối đối với bộ ảnh phức tạp như STL10. Do đó, PuVAE chưa phù hợp cho các tập dữ liệu lớn.

Tài liệu

- [1] Naveed Akhtar **and** Ajmal Mian. “Threat of adversarial attacks on deep learning in computer vision: A survey”. **in***Ieee Access*: 6 (2018), **pages** 14410–14430.
- [2] Martin Arjovsky, Soumith Chintala **and** Léon Bottou. *Wasserstein GAN*. 2017. arXiv: [1701.07875](https://arxiv.org/abs/1701.07875) [stat.ML]. URL: <https://arxiv.org/abs/1701.07875>.
- [3] Nicholas Carlini **and** David Wagner. “Towards evaluating the robustness of neural networks”. **in***2017 IEEE Symposium on Security and Privacy (SP)*: Ieee. 2017, **pages** 39–57.
- [4] Carl Doersch. “Tutorial on variational autoencoders”. **in***arXiv preprint arXiv:1606.05908*: (2016).
- [5] Ian Goodfellow **and others**. “Generative adversarial nets”. **in***Advances in neural information processing systems*: 27 (2014).
- [6] Ian J Goodfellow. “Explaining and harnessing adversarial examples”. **in***arXiv preprint arXiv:1412.6572*: (2014).
- [7] Uiwon Hwang **and others**. “Puvae: A variational autoencoder to purify adversarial examples”. **in***IEEE Access*: 7 (2019), **pages** 126582–126593.
- [8] Diederik P Kingma. “Auto-encoding variational bayes”. **in***arXiv preprint arXiv:1312.6114*: (2013).
- [9] Alex Krizhevsky, Geoffrey Hinton **and others**. “Learning multiple layers of features from tiny images”. **in**(2009).
- [10] Alexey Kurakin, Ian J Goodfellow **and** Samy Bengio. “Adversarial examples in the physical world”. **in***Artificial intelligence safety and security*: Chapman **and** Hall/CRC, 2018, **pages** 99–112.
- [11] Yann LeCun **and others**. “Gradient-based learning applied to document recognition”. **in***Proceedings of the IEEE*: 86.11 (1998), **pages** 2278–2324.
- [12] Dongyu Meng **and** Hao Chen. “Magnet: a two-pronged defense against adversarial examples”. **in***Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*: 2017, **pages** 135–147.
- [13] P Samangouei. “Defense-gan: protecting classifiers against adversarial attacks using generative models”. **in***arXiv preprint arXiv:1805.06605*: (2018).
- [14] Florian Tramèr **and others**. “Ensemble adversarial training: Attacks and defenses”. **in***arXiv preprint arXiv:1705.07204*: (2017).
- [15] Han Xiao, Kashif Rasul **and** Roland Vollgraf. “Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms”. **in***arXiv preprint arXiv:1708.07747*: (2017).